

## 7.1 Scaling Laws

Kaplan power laws, then Chinchilla: GPT-3 was 1.7 tokens/parameter, not 20.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Bigger models, trained on more data with more compute, tend to get better at next-token prediction.

**Scaling laws** make that trend a curve you can fit, not a slogan. They do not promise a better chatbot; they promise a lower **pretraining loss** if the recipe stays the same.

**First time this method appears.** A **scaling law** is a fitted curve of pretraining loss versus scale, not a promise of a better chatbot.

**What.** Kaplan: loss vs compute (and  $N, D$ ) looks like a power law plus a floor. Chinchilla: at fixed FLOPs, grow **tokens and parameters together** ( $\sim 20$  tokens/parameter in that paper). **Why.** “We trained a 7B” is not a complete sentence. You need the token budget and whether you were compute-optimal. **Architecture.** Not a new net. A log-log plot and a recipe  $(N, D, C)$ . **How.** Fit  $L \approx aC^{-b} + L_\infty$  (Lab 7 algebra). Then do Chinchilla arithmetic:  $7B \times 20 = 140B$  tokens. **Formula.**  $L(C) \approx aC^{-b} + L_\infty$ . Training FLOPs cartoon:  $\approx 6ND$ . **Tradeoffs.** + Planning a run. –  $L$  is not chat quality; data mixture and alignment sit off the curve; four toy points are not a law.

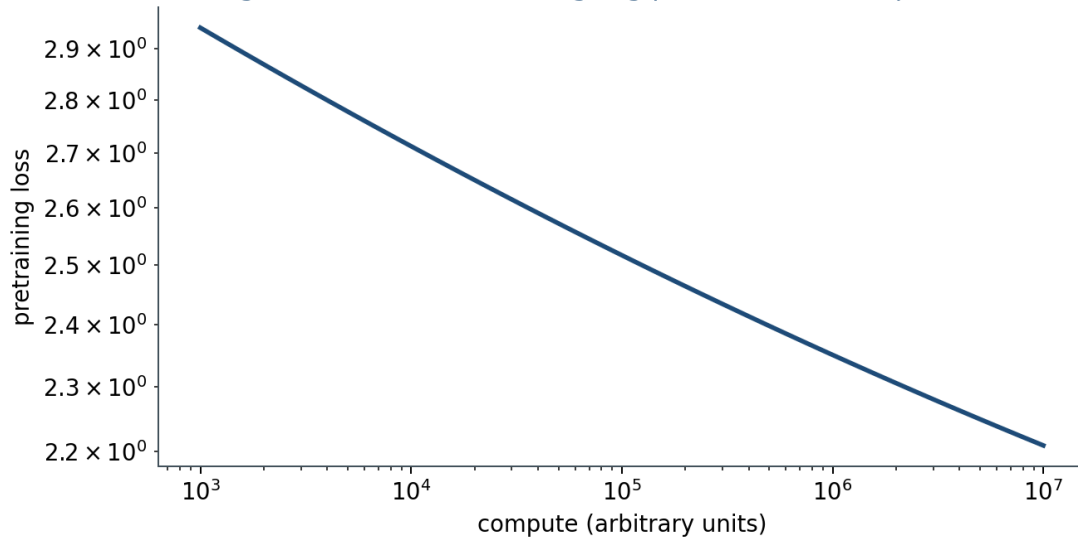
### 1. Kaplan: loss as a power of compute

Kaplan et al. (2020) fit language-model cross-entropy against compute  $C$  (and against parameters  $N$ , dataset size  $D$ ) and saw smooth **power laws** over many orders of magnitude:

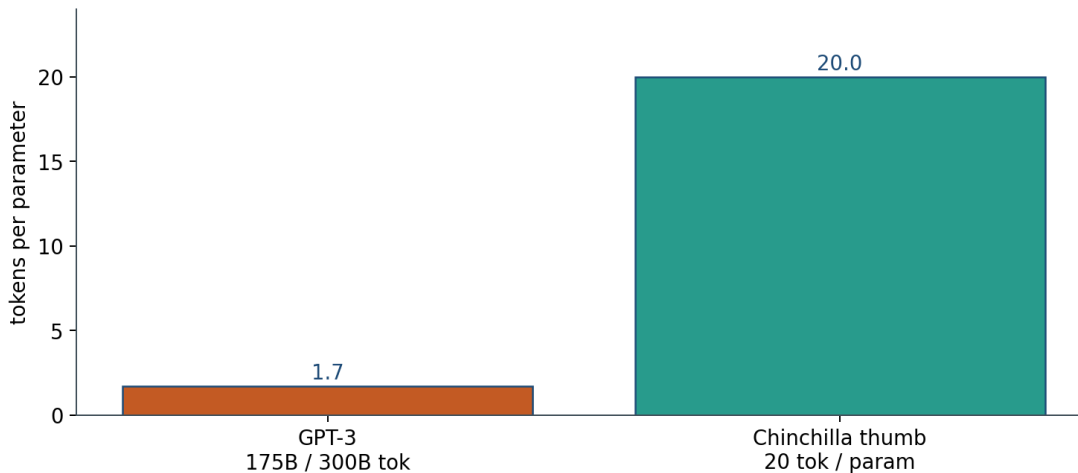
$$L(C) \approx aC^{-b} + L_\infty.$$

The irreducible term  $L_\infty$  is entropy of the data plus whatever the architecture cannot represent. The interesting bit is the slope  $-b$ : extra FLOPs still buy loss, with diminishing returns.

Scaling laws: smoother on a log-log plot, until data or params starve



CS224N L9: GPT-3 was underfed tokens for its size.

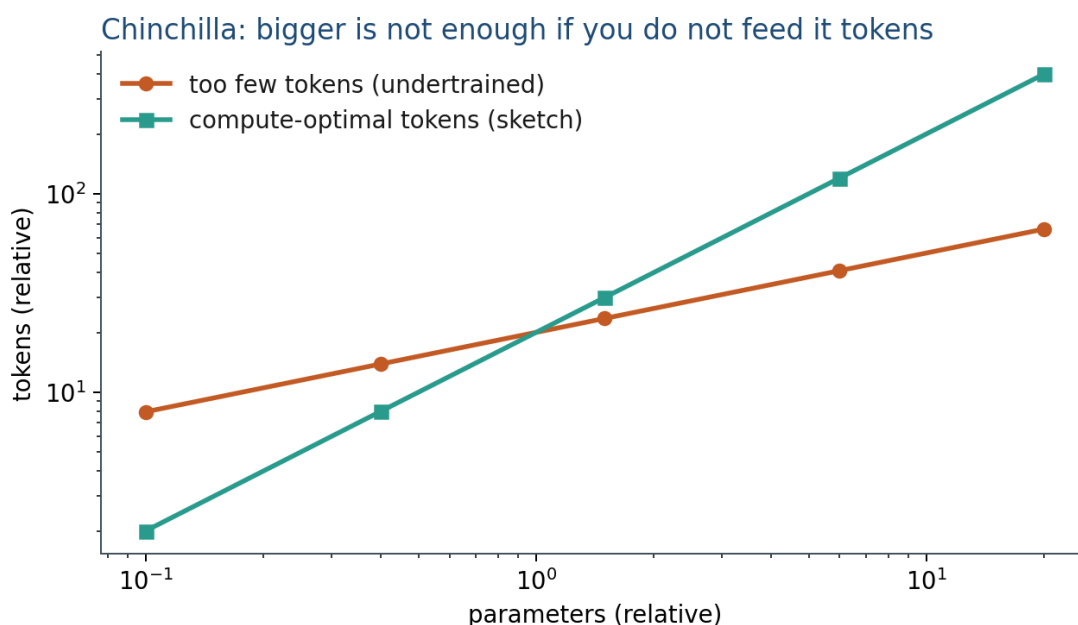


Fix two of  $\{N, D, C\}$  and the third is constrained. Early GPT-3-style runs were **compute-optimal for a short training budget** that underused data: a large  $N$ , not enough tokens.

Lab 7 will fit  $L = aC^{-b} + c$  on constructed points so you see the algebra, not a GPU cluster.

## 2. Chinchilla: tokens versus parameters

Hoffmann et al. (2022), **Chinchilla**: for a fixed FLOP budget, you should scale **tokens and parameters together**, not dump the budget into a huge model that sees the data once. Their fit said, roughly, equal scaling and about **20 tokens per parameter** for the models they trained.



A smaller model trained longer can beat a larger undertrained one. That is why later LLaMA-style recipes look “small” next to GPT-3 and still read well: they were fed more tokens.

Stanford CS224N 2025 L9’s classroom example: GPT-3 was 175  $B$  parameters on 300  $B$  tokens, about 1.7 tokens per parameter. Chinchilla’s rule of thumb is nearer 20. Same lecture: a 70  $B$  model trained with enough tokens beat much larger undertrained nets.

A 7B model at 20 tokens/parameter wants about  $7 \times 10^9 \times 20 = 1.4 \times 10^{11}$  tokens. If you only have 14B tokens, you are  $10 \times$  short of that rule of thumb—not “we scaled.”

Kaplan-style runs often **oversize**  $N$  for a short  $C$ : the compute is spent on a wide net that barely sees  $D$ . Chinchilla re-spends that  $C$  on more tokens. Same FLOP budget, different  $(N, D)$ . Write both numbers, not only “7B.”

### 3. What a law does not tell you

Downstream accuracy is not  $L$ . Alignment (Weeks 8–9), data quality, mixture, and context length all sit off the curve. Inference cost scales with  $N$  even if you already paid for training. For a project: cite the checkpoint size and the token budget if you know them; do not claim “we scaled” because you trained two extra epochs on a toy set.

### 4. Teaching this note

About **35 minutes** at the board: write  $L(C)$ , mark  $L_\infty$ , then a Chinchilla arithmetic example (tokens vs parameters at fixed FLOPs). Play Karpathy **after 20:00**, especially **25:43–27:43** (scaling laws, ~2 min, plus a little context). The fit in Lab 7 is later in the week.

Put 20 tokens/parameter on the board as a **rule of thumb from one paper**, not a law of nature. Then do the  $7B \times 20$  arithmetic so “we trained a 7B” is never a complete sentence.

---

## 5. Worked example

Suppose  $L = 2C^{-0.15} + 1.4$ . At  $C = 10^4$ ,

$$L \approx 2 \cdot 10^{-0.6} + 1.4 \approx 2 \cdot 0.251 + 1.4 \approx 1.90.$$

At  $C = 10^6$ ,  $10^{-0.9} \approx 0.126$ , so  $L \approx 2 \cdot 0.126 + 1.4 = 1.65$ . A  $100\times$  compute jump bought 0.25 nats, not a new architecture. If your measured loss is already 1.42, you are sitting on  $L_\infty$ : extra FLOPs buy almost nothing.

Chinchilla, same FLOPs. Training FLOPs scale roughly as  $6ND$  (forward+backward). Model A:  $N = 4N_0$ ,  $D = D_0 / 4$ . Model B:  $N = N_0$ ,  $D = D_0$ . Same  $ND$ . Chinchilla picks the **balanced** pair, not the  $4\times$ -larger starved model. If the rule of thumb is 20 tokens/parameter, B matches it when  $D_0 = 20N_0$ ; A has only 5 tokens/parameter.

---

## 6. Where students get stuck

- Reading  $L$  as “chat quality.” It is pretraining cross-entropy.
  - Thinking a bigger  $N$  always wins at fixed FLOPs. Chinchilla is the counterexample.
  - Fitting  $a, b, c$  on four noisy toy points and claiming a law. Lab 7 is the algebra; the paper’s range is many orders of magnitude.
- 

## 7. Video

[Karpathy — Intro to Large Language Models](#). Play from **20:00**, and pause on **25:43–27:43** (scaling laws). The first 20 min were Week 1; this hour is the compute curve and why tokens matter as much as parameters.

---

## 8. Practice

1. Two models, same FLOPs. One has  $4\times$  the parameters and  $1/4$  the tokens of the other. Who does Chinchilla pick, and why?
2. If  $L_\infty$  is already almost your measured loss, what does extra compute buy?
3. Why can a scaling law look great while a chat eval is flat?
4. Using  $L = 2C^{-0.15} + 1.4$ , compute  $L$  at  $C = 10^5$ . ( $10^{-0.75} \approx 0.178$ .)

5. A 1B-parameter model trained on 4B tokens. Using 20 tokens/parameter, is it over- or under-trained on data? By what factor?